AWS NETWORKING

STUDENT PROJECT REPORT / PROJECT 01

BUILD / TEST / EXPLAIN

AWS

VPC Peering

Connect two VPCs.
Prove private connectivity.

EC2, custom route tables, security groups and a bastion host

NAME	**Glen Nitish**
BATCH	**AWS INTERNSHIP BATCH 1**
PROJECT	**AWS VPC Peering (Project 01)**
REGION	**US East (N. Virginia), us-east-1**
COMPLETION DATE	**29 October 2026**

2 VPCs / 3 EC2 instances / 1 private connection

1. Project overview

The goal of this project was to build a working private network connection between two VPCs and to explain why every route and security rule is needed. An application server (App-A) lives in VPC A and a second workload (App-B) lives in VPC B. Both servers have no public IP address. They exchange traffic only over private IPv4 addresses through a VPC peering connection. One public bastion host in VPC A is used for administration.

What was demonstrated

- Non-overlapping VPC and subnet CIDRs: 10.10.0.0/16 and 10.20.0.0/16.
- Public and private subnets with explicit route table associations.
- An active peering connection, forward and return routes, and security group rules that allow only the required inbound traffic.
- Successful two-way ping, private SSH to App-B through the bastion, and a controlled failure followed by recovery.
- Complete cleanup of every resource created for the lab.

Item	Value
Region / Availability Zone	us-east-1 (N. Virginia) / us-east-1c
Operating system / instance type	Amazon Linux 2023 / t3.micro
Key pair	glens-lab (RSA, .pem) - kept on my laptop only, never copied to any instance
Administration path	Laptop > bastion > App-A or App-B using SSH ProxyJump
Project traffic	App-A and App-B over private IPs through the peering connection

2. Architecture and address plan

One internet entry point (the bastion in the public subnet of VPC A), two private workloads and a direct peering connection between the VPCs. VPC B has no internet gateway, and peering does not give VPC B internet access through VPC A.

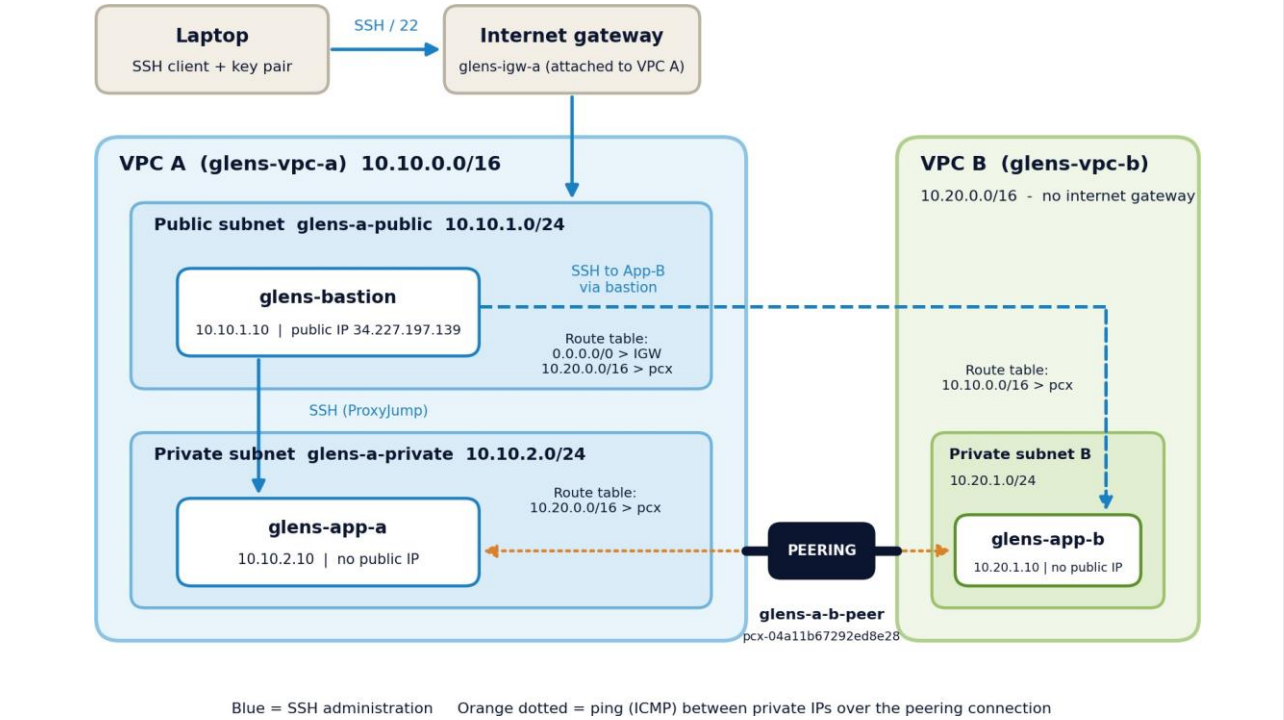


Figure 1. *Architecture diagram of the network that was built (names and IDs are the real resources used).*

Resource inventory

Resource	Name	CIDR / private IP	Resource ID
VPC A	glens-vpc-a	10.10.0.0/16	vpc-0847b1f0a36e6ac50
Public subnet (VPC A)	glens-a-public	10.10.1.0/24	subnet-0483a35933b62cf9d
Private subnet (VPC A)	glens-a-private	10.10.2.0/24	subnet-0ce521594d99a84d3
VPC B	glens-vpc-b	10.20.0.0/16	vpc-017a3a322cad97920
Private subnet (VPC B)	glens-b-private	10.20.1.0/24	subnet-0f35e94875ff2e14b
Internet gateway	glens-igw-a	Attached to VPC A	igw-02aeacfcf52f7156a
Public route table	glens-rt-a-public	Subnet: glens-a-public	rtb-0f940530014f419a4
Private route table A	glens-rt-a-private	Subnet: glens-a-private	rtb-03cbf482a96428dfb
Private route table B	glens-rt-b-private	Subnet: glens-b-private	rtb-0eae10149051581f9
Peering connection	glens-a-b-peer	VPC A to VPC B	pcx-04a11b67292ed8e28
Security group (bastion)	glens-sg-bastion	VPC A	sg-09e1425da73cacaf8
Security group (App-A)	glens-sg-app-a	VPC A	sg-0ac149897eb814386
Security group (App-B)	glens-sg-app-b	VPC B	sg-0f6e363d21d0feb80
Bastion instance	glens-bastion	10.10.1.10 + public IPv4 34.227.197.139	i-0356cfdddb3bcb5ea
App-A instance	glens-app-a	10.10.2.10 (no public IP)	i-0a78ea4d6eaac4145
App-B instance	glens-app-b	10.20.1.10 (no public IP)	i-004b461a70d272c87

Why these CIDRs: 10.10.0.0/16 and 10.20.0.0/16 do not overlap, so a peering connection can be created. Each /24 subnet sits inside its own VPC range. Different subnet ranges alone would not fix overlapping VPC CIDRs.

3. VPCs, subnets and internet gateway

VPC A (10.10.0.0/16) and VPC B (10.20.0.0/16) were created with the VPC-only option so every subnet and route is controlled manually. VPC A has two /24 subnets in us-east-1c, and VPC B has one.

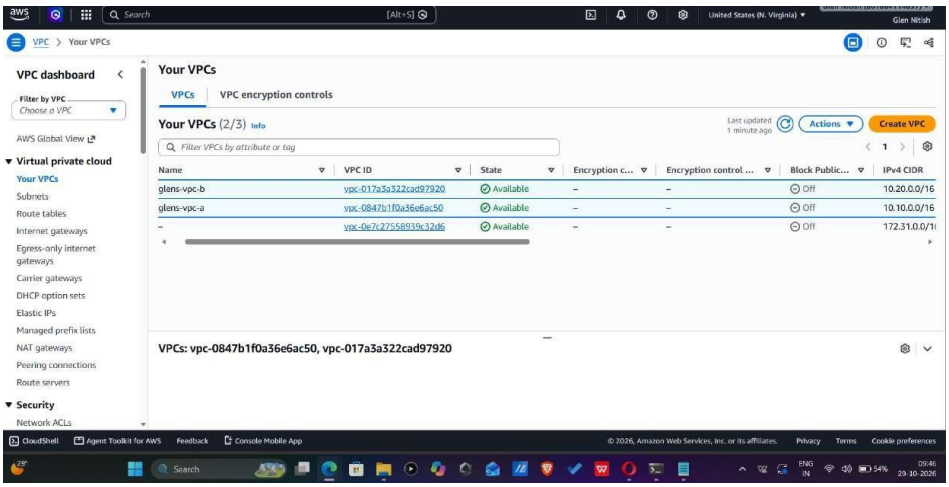


Figure 2. *VPC console - glens-vpc-a (10.10.0.0/16) and glens-vpc-b (10.20.0.0/16) are Available next to the default VPC.*

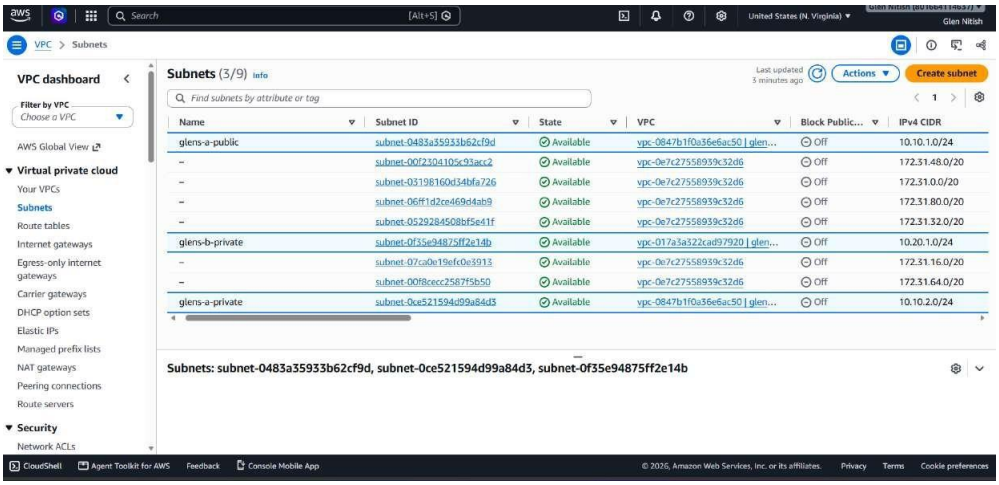


Figure 3. *Subnets - glens-a-public (10.10.1.0/24) and glens-a-private (10.10.2.0/24) in VPC A, glens-b-private (10.20.1.0/24) in VPC B.*

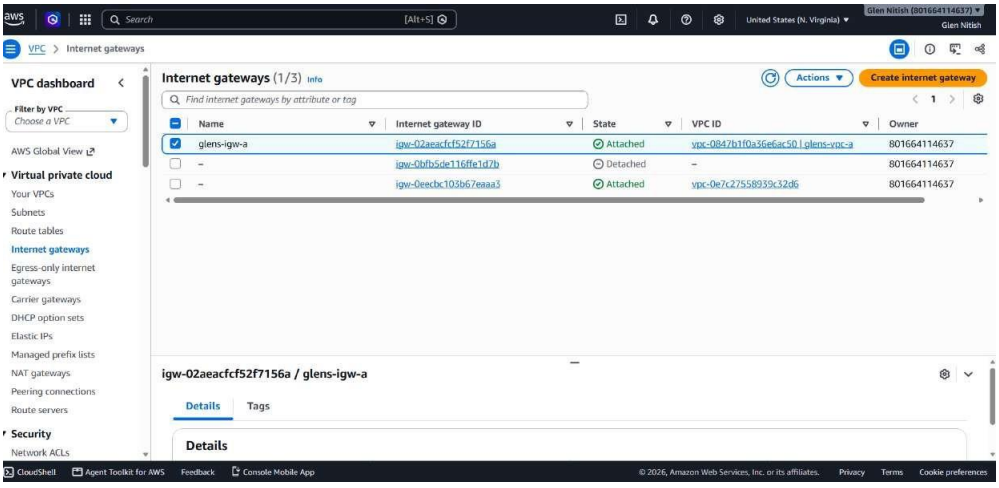


Figure 4. *Internet gateway glens-igw-a (igw-02aeacfcf52f7156a) is Attached to vpc-0847b1f0a36e6ac50 (glens-vpc-a). VPC B has no internet gateway.*

4. Route tables and subnet associations

Each subnet is explicitly associated with its own route table. The public route table has the default route to the internet gateway. Both private route tables have no internet route; after peering they each received one route to the other VPC.

Route table	Associated subnet	Destination	Target	Purpose
glens-rt-a-public	glens-a-public	10.10.0.0/16	local	Bastion to App-A
		0.0.0.0/0	igw-02aeacfcf52f7156a	Bastion internet access
		10.20.0.0/16	pcx-04a11b67292ed8e28	Bastion SSH to App-B
glens-rt-a-private	glens-a-private	10.10.0.0/16	local	Existing VPC A route
		10.20.0.0/16	pcx-04a11b67292ed8e28	App-A to VPC B (forward path)
glens-rt-b-private	glens-b-private	10.20.0.0/16	local	Existing VPC B route
		10.10.0.0/16	pcx-04a11b67292ed8e28	Return path to VPC A

Public route table (VPC A)

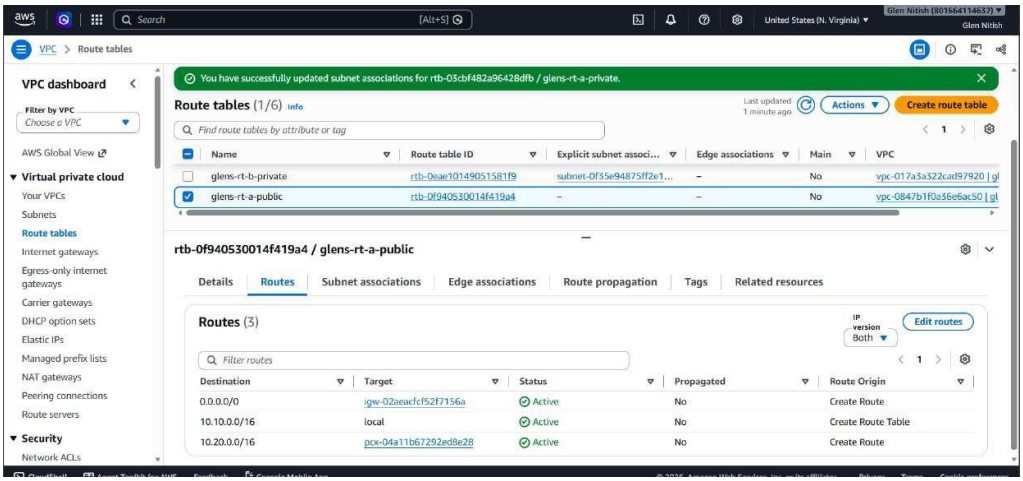


Figure 5. *glens-rt-a-public - routes: 0.0.0.0/0 to the internet gateway, 10.10.0.0/16 local, 10.20.0.0/16 to the peering connection (all Active).*

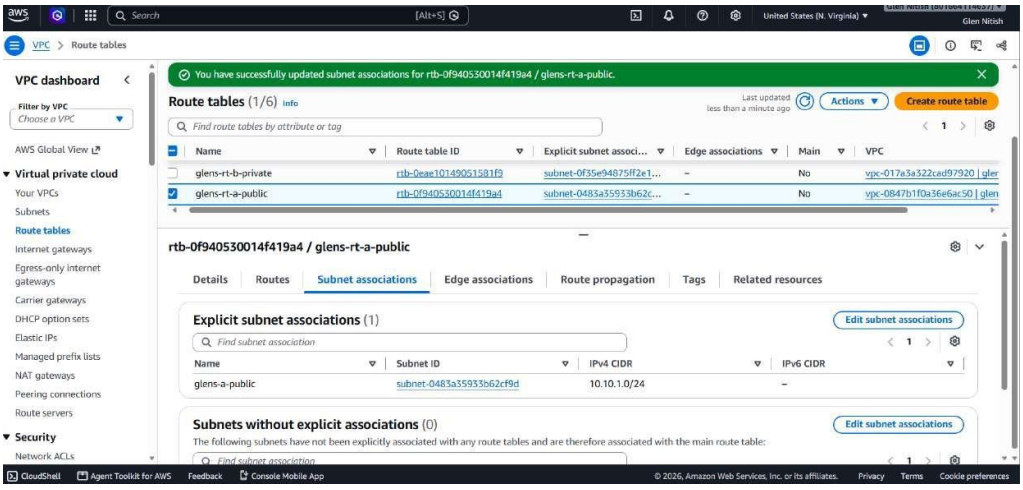


Figure 6. *glens-rt-a-public - explicit subnet association: glens-a-public (10.10.1.0/24).*

Private route table (VPC A)

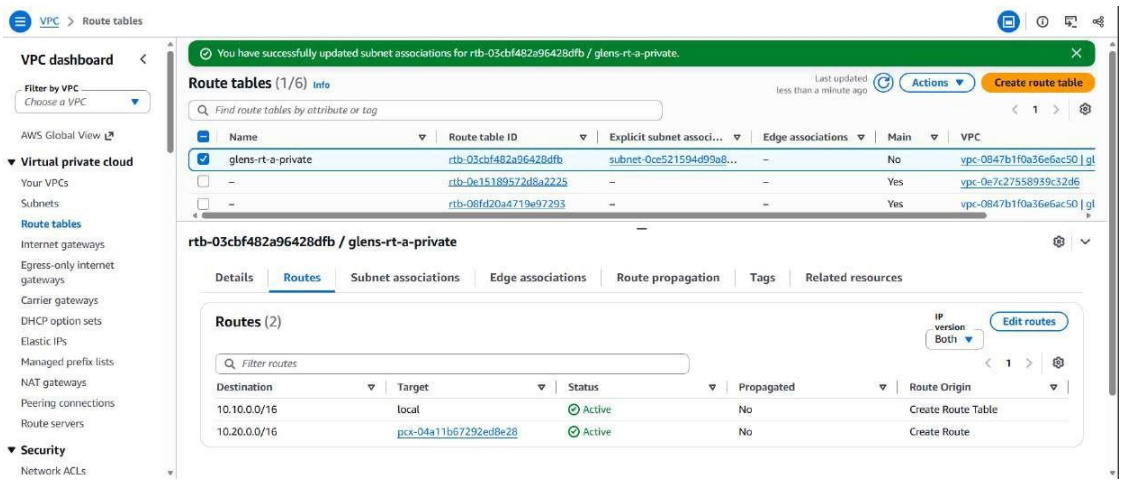


Figure 7. *glens-rt-a-private - routes: 10.10.0.0/16 local and 10.20.0.0/16 to the peering connection (Active). No internet route.*

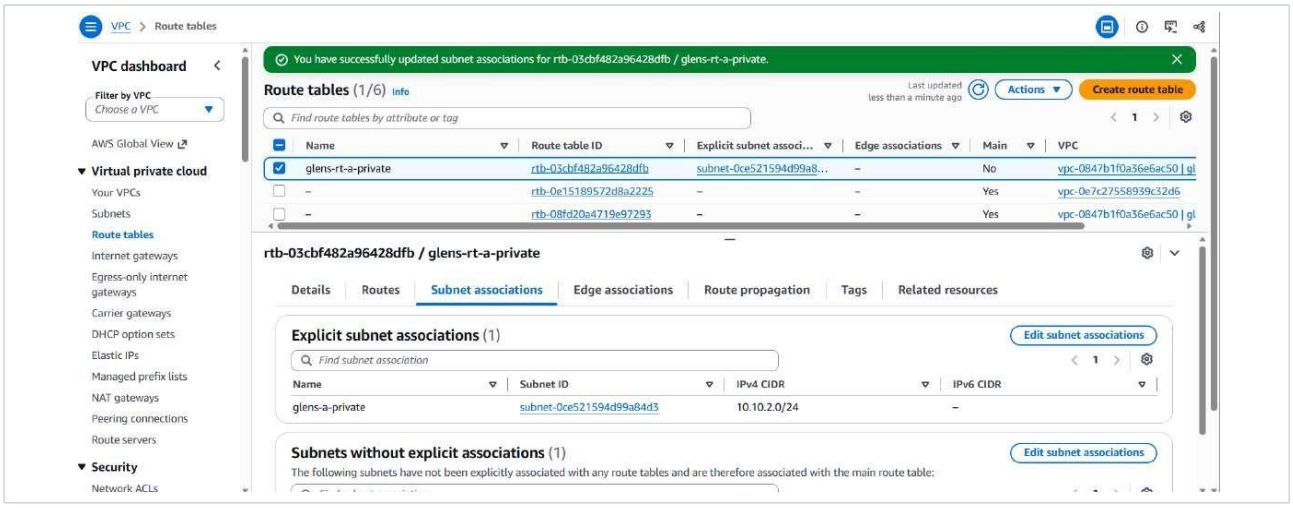


Figure 8. *glens-rt-a-private - explicit subnet association: glens-a-private (10.10.2.0/24).*

Private route table (VPC B)

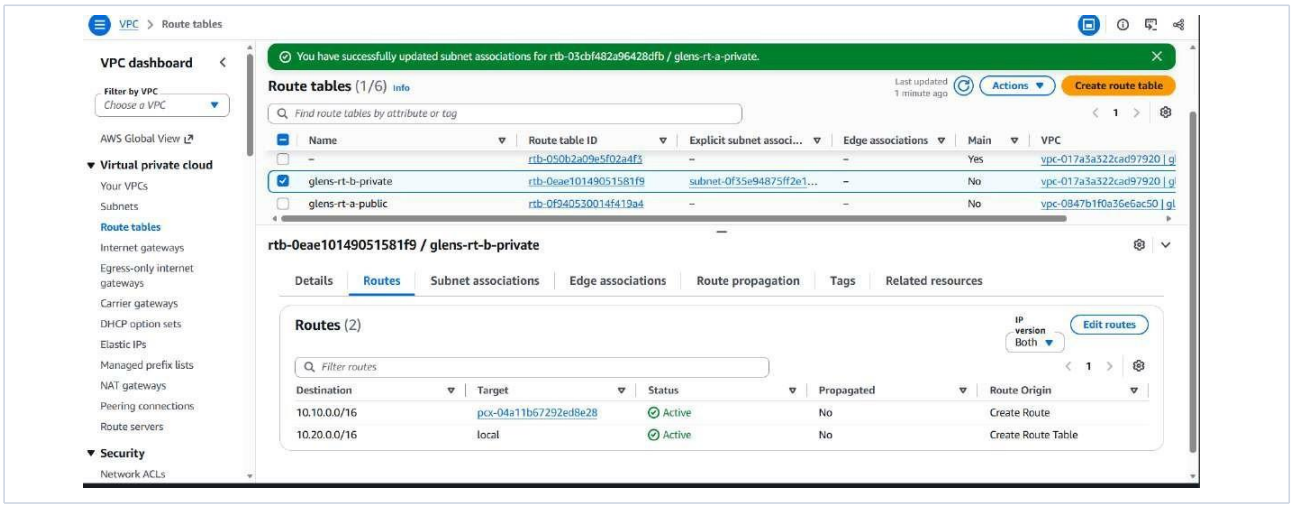


Figure 9. *glens-rt-b-private - routes: 10.20.0.0/16 local and 10.10.0.0/16 to the peering connection (Active) - the return path.*

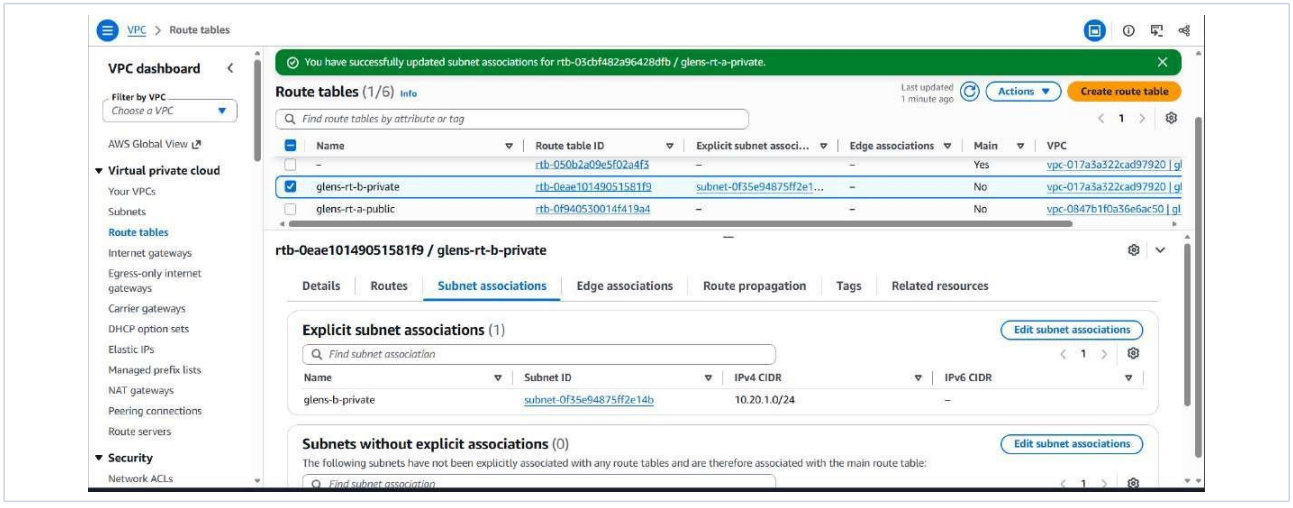


Figure 10. *glens-rt-b-private - explicit subnet association: glens-b-private (10.20.1.0/24).*

> **Why the public subnet needs a peering route:** with ProxyJump the bastion opens its own SSH connection to App-B, so the bastion subnet must know how to reach VPC B. The more specific 10.20.0.0/16 route is selected instead of the 0.0.0.0/0 internet route.

5. Security group rules

Security groups are stateful, so replies to permitted traffic are allowed automatically. Outbound rules were left as the default allow-all. SSH and ICMP are not open to 0.0.0.0/0.

Security group	Type / port	Source	Reason
glens-sg-bastion (VPC A)	SSH / TCP 22	157.51.0.0/16 (my network's public range)	Only my network can reach the bastion
glens-sg-app-a (VPC A)	SSH / TCP 22	10.10.1.10/32 (bastion)	Admin access only through the bastion
glens-sg-app-a (VPC A)	Custom ICMP IPv4 - Echo Request	10.20.1.10/32 (App-B)	Ping from App-B to App-A
glens-sg-app-b (VPC B)	SSH / TCP 22	10.10.1.10/32 (bastion)	Admin access only through the bastion
glens-sg-app-b (VPC B)	Custom ICMP IPv4 - Echo Request	10.10.2.10/32 (App-A)	Ping from App-A to App-B

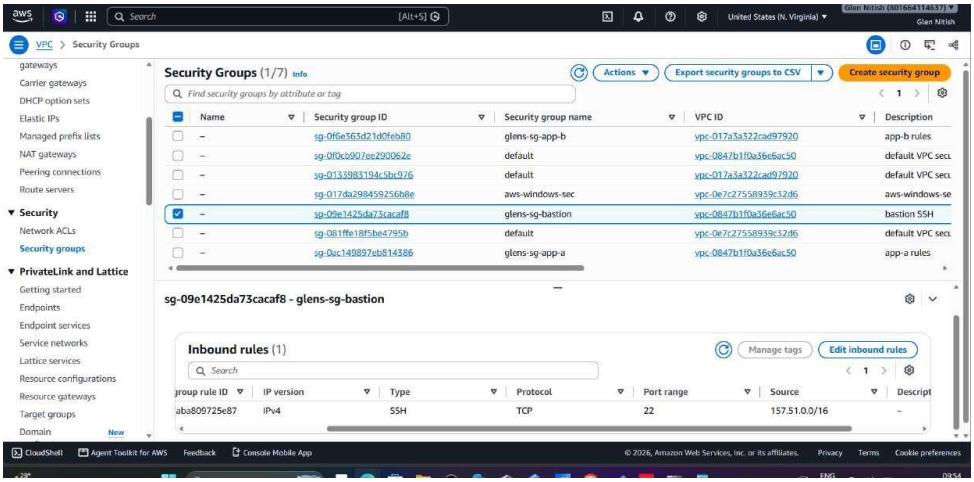


Figure 11. *glens-sg-bastion (sg-09e1425da73cacaf8) - inbound rule: SSH, TCP 22, from my network range 157.51.0.0/16.*

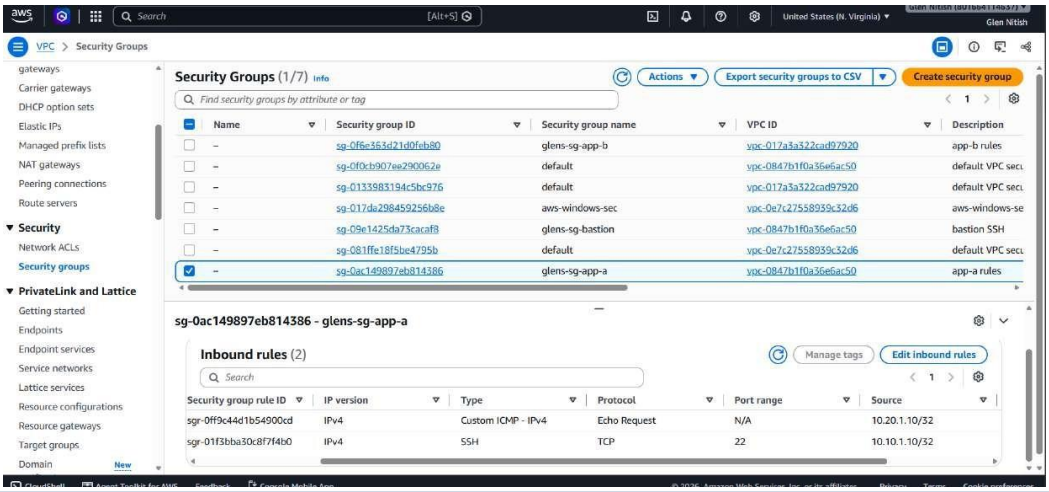


Figure 12. *glens-sg-app-a (sg-0ac149897eb814386) - inbound rules: ICMP Echo Request from 10.20.1.10/32 and SSH from 10.10.1.10/32.*

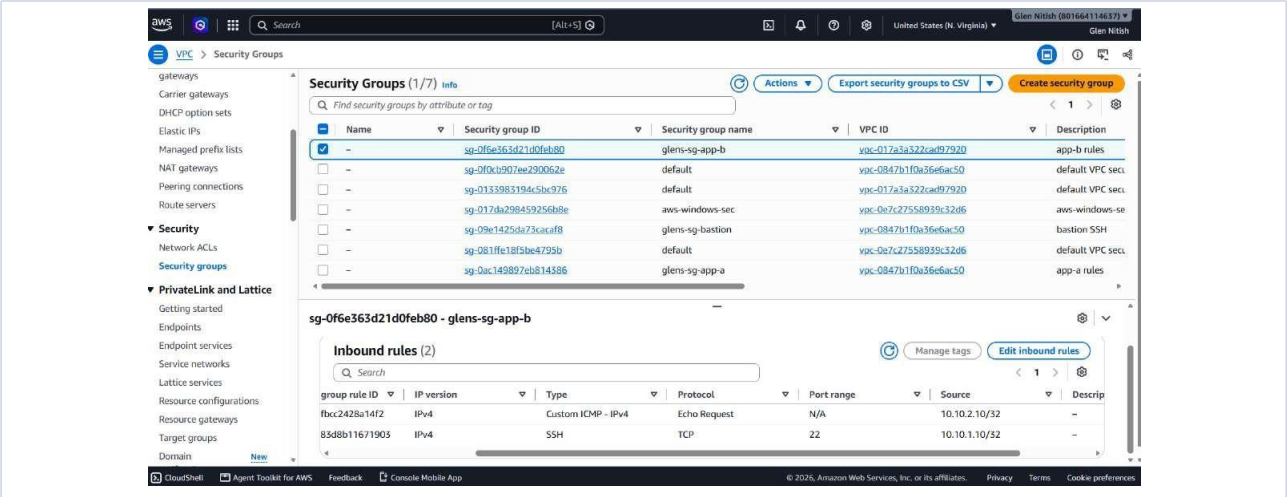


Figure 13. glens-sg-app-b (sg-0f6e363d21d0feb80) - inbound rules: ICMP Echo Request from 10.10.2.10/32 and SSH from 10.10.1.10/32.

> **Port and source are different fields.** SSH uses port 22; the private IP goes in Source. App-B sees the bastion's private IP (10.10.1.10) as the SSH source because the bastion opens the connection when ProxyJump is used.

6. EC2 instances

Three Amazon Linux 2023 t3.micro instances were launched with fixed private IPs. Only the bastion has a public IPv4 address. All three passed their 3/3 status checks.

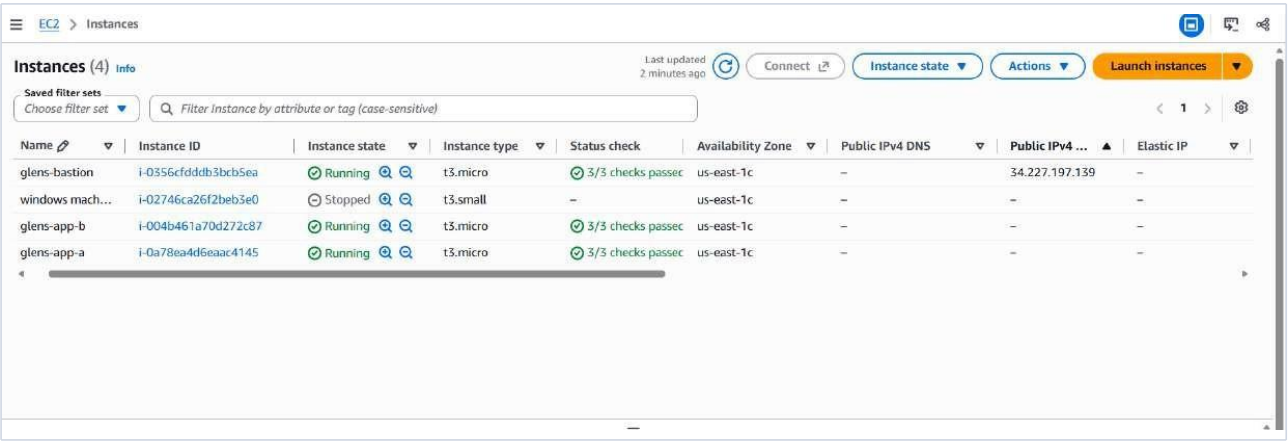


Figure 14. EC2 console - glens-bastion, glens-app-b and glens-app-a are Running with 3/3 checks passed in us-east-1c. Only the bastion has a public IPv4 (34.227.197.139).

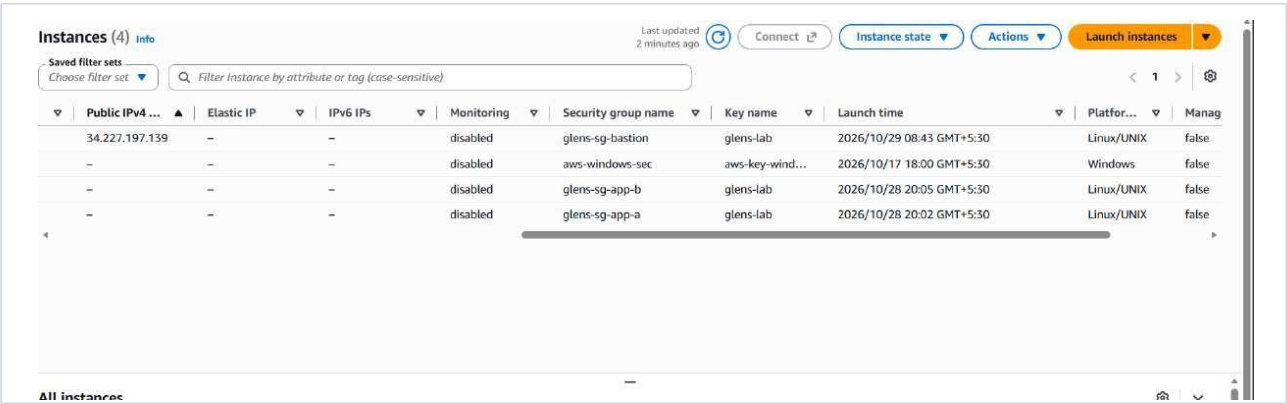


Figure 15. EC2 console (right-hand columns) - security groups glens-sg-bastion, glens-sg-app-b and glens-sg-app-a, and key pair glens-lab. App-A and App-B show no public IPv4.

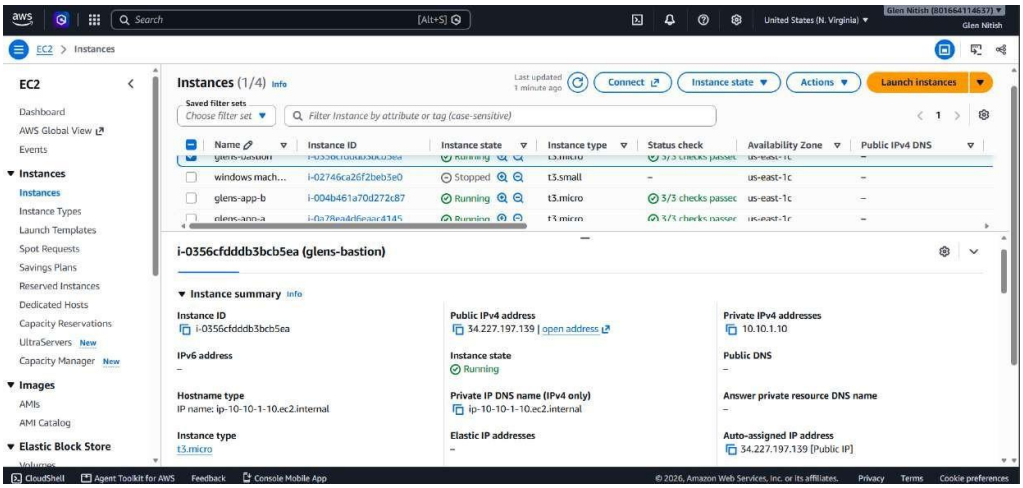


Figure 16. *glens-bastion (i-0356cfdddb3bcb5ea) - private IPv4 10.10.1.10, public IPv4 34.227.197.139.*

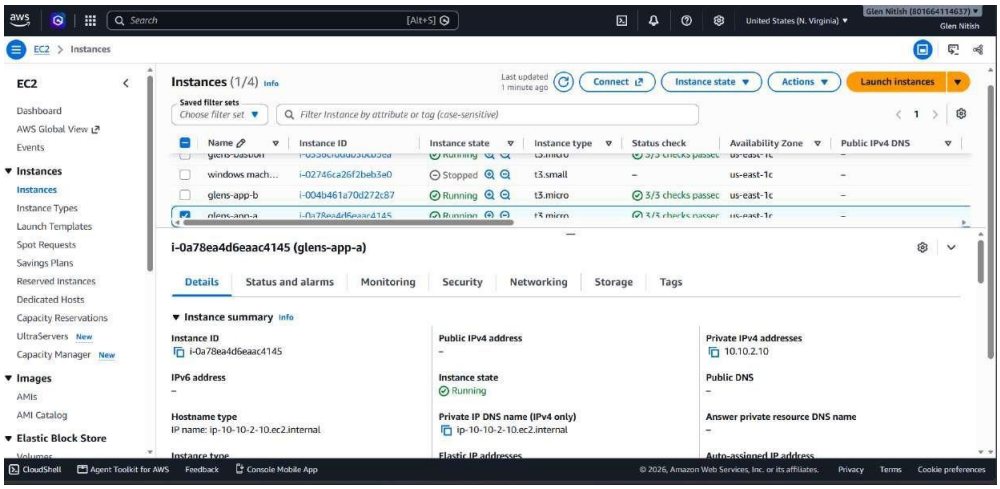


Figure 17. *glens-app-a (i-0a78ea4d6eaac4145) - private IPv4 10.10.2.10, Public IPv4 address empty (Test 05, isolation).*

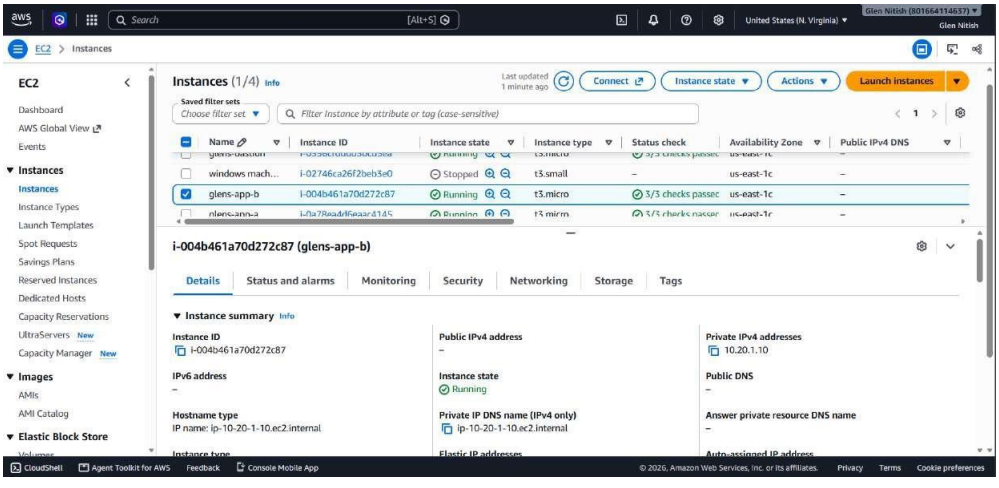


Figure 18. *glens-app-b (i-004b461a70d272c87) - private IPv4 10.20.1.10, Public IPv4 address empty (Test 05, isolation).*

7. VPC peering connection

The peering request was created from VPC A (requester) to VPC B (accepter) in the same account and Region, then accepted. Accepting is required even when both VPCs belong to the same account.

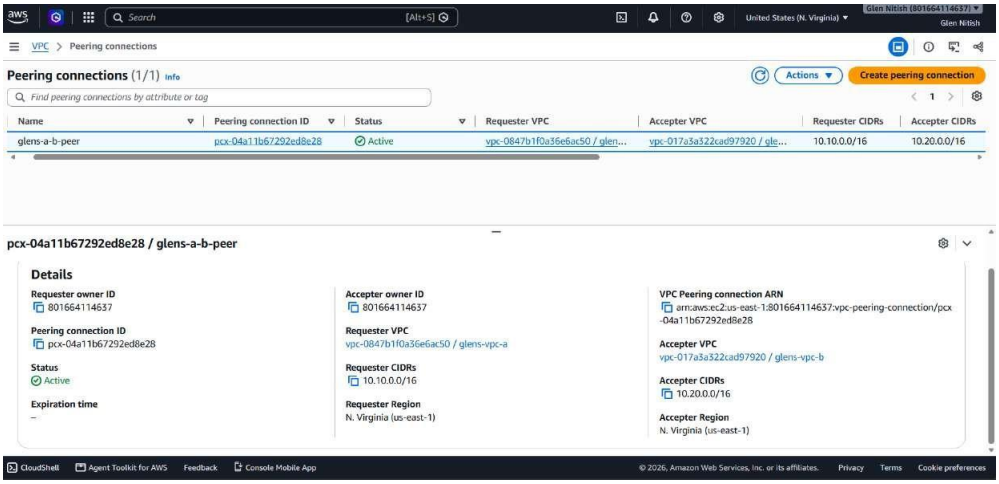


Figure 19. *Peering connection glens-a-b-peer (pcx-04a11b67292ed8e28) - Status Active. Requester: vpc-0847b1f0a36e6ac50 / glens-vpc-a (10.10.0.0/16). Accepter: vpc-017a3a322cad97920 / glens-vpc-b (10.20.0.0/16). Both in us-east-1.*

> Active peering alone does not make ping or SSH work. Traffic only flows once both route tables point to the peering connection and the security groups allow the traffic.

8. Connectivity tests

All tests use private destination IPs and were run from my laptop through the bastion with SSH ProxyJump. The key stays on my laptop.

Test	Where / command	Expected	Actual result
02 / Forward path	App-A: ping -c 4 10.20.1.10	4 replies, 0% loss	4 transmitted, 4 received, 0% packet loss - PASS
03 / Reverse path	App-B: ping -c 4 10.10.2.10	4 replies, 0% loss	4 transmitted, 4 received, 0% packet loss - PASS
04 / Private SSH	Laptop: ssh glens-app-b	Shell on App-B through bastion	Shell opened on ip-10-20-1-10 - PASS
05 / Isolation	EC2 instance details	No public IPv4 on App-A / App-B	Public IPv4 address empty on both - PASS

Before peering and the peer routes were added, VPC A's route tables contained only the local 10.10.0.0/16 route. There was no route to 10.20.0.0/16 and no peering connection, so traffic from App-A to App-B had no path. The failure screenshots in section 9 show how the same ping behaves (100% packet loss) when a required layer is missing.

Test 02 - forward path (App-A to App-B)

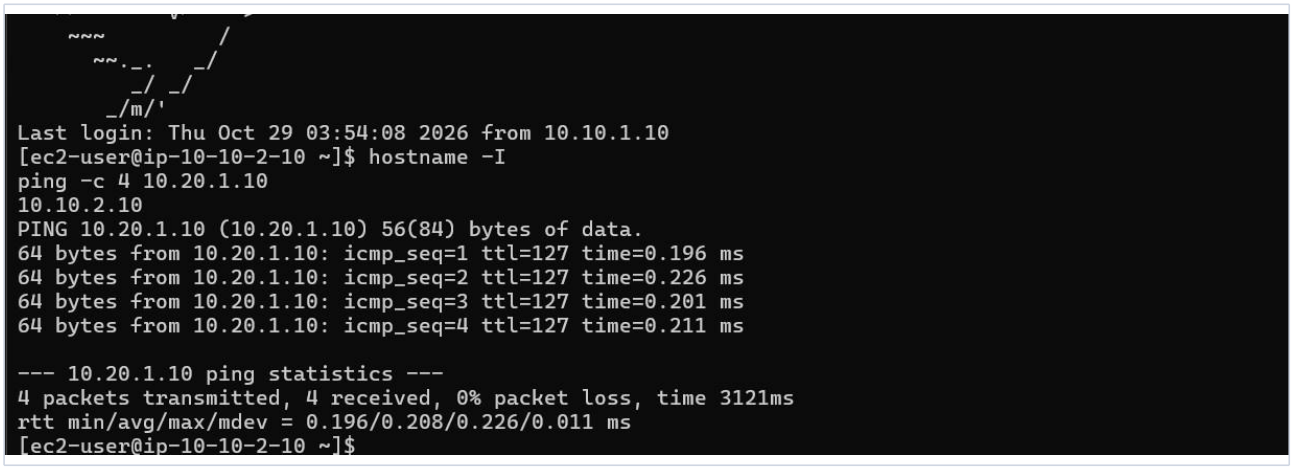


Figure 20. *Host: App-A (10.10.2.10, confirmed with hostname -I) | Test 02: ping -c 4 10.20.1.10 | Result: 4 packets transmitted, 4 received, 0% packet loss.*

Test 03 - reverse path (App-B to App-A)

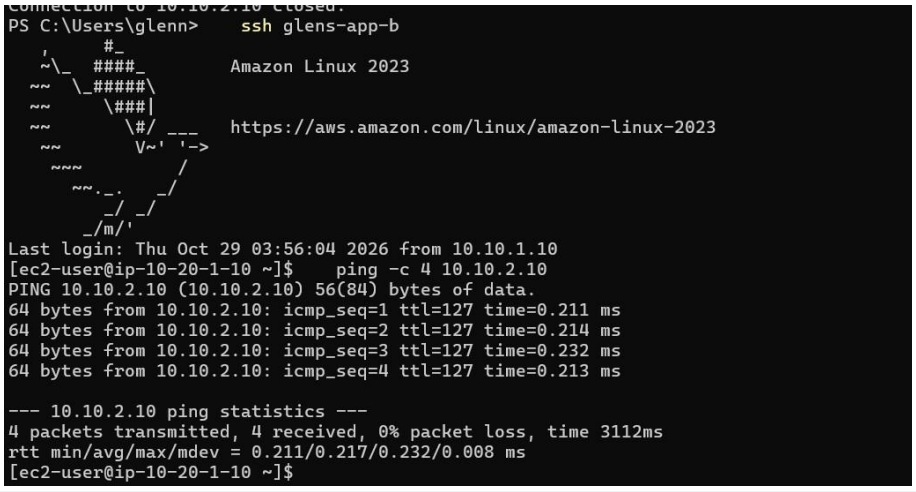


Figure 21. *Host: App-B (ip-10-20-1-10) | Test 03: ping -c 4 10.10.2.10 | Result: 4 packets transmitted, 4 received, 0% packet loss.*

Test 04 - private SSH to App-B through the bastion

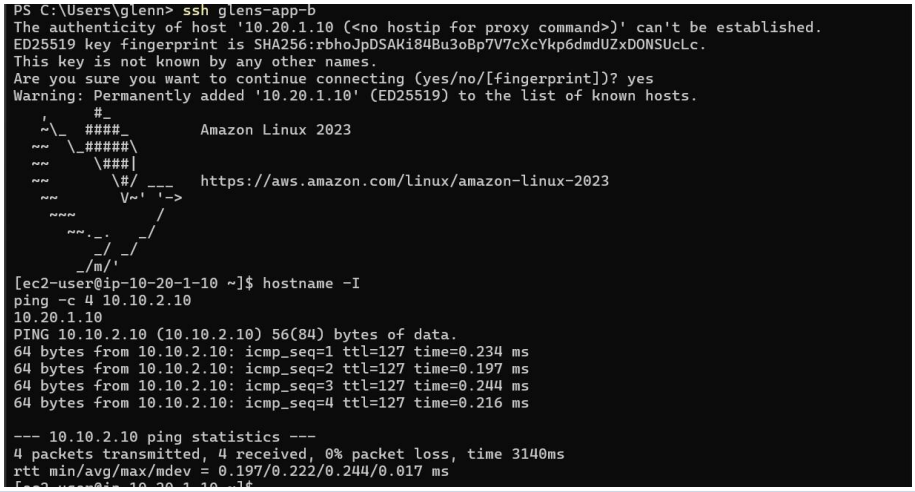


Figure 22. *Host: laptop, then App-B | Test 04: ssh glens-app-b (ProxyJump through the bastion) | Result: shell on App-B (hostname -I shows 10.20.1.10), and ping to 10.10.2.10 gives 0% packet loss.*

9. Controlled failure and recovery

To show that security rules matter as much as routes, I temporarily removed only the App-B inbound ICMP Echo Request rule (source 10.10.2.10/32) and repeated the ping from App-A. The SSH rule stayed in place. After restoring the exact rule the ping worked again.

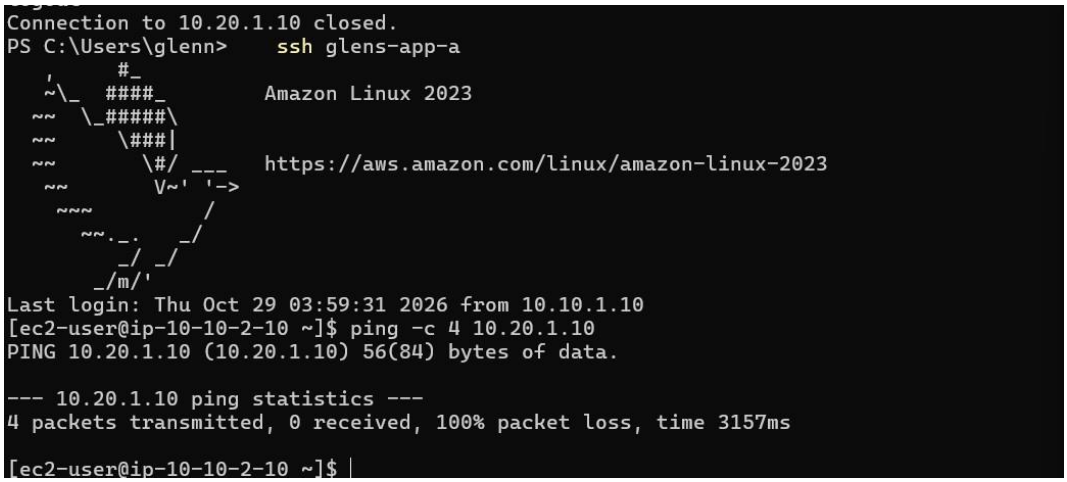


Figure 23. *Host: App-A | Failure test: ping -c 4 10.20.1.10 with the App-B ICMP rule removed | Result: 4 transmitted, 0 received, 100% packet loss.*

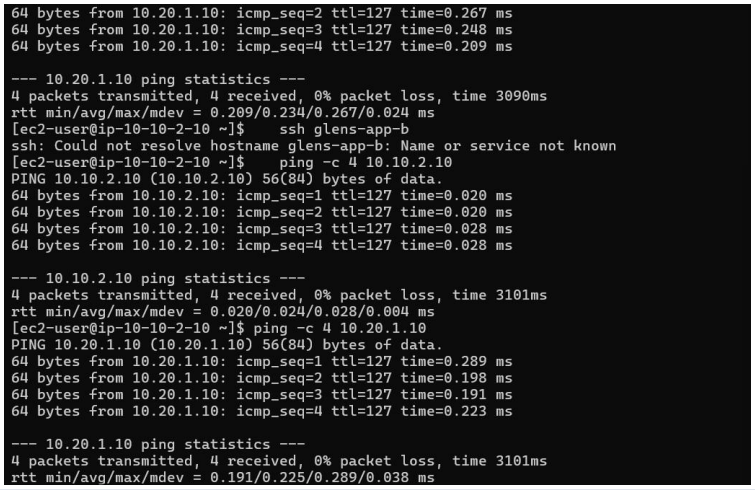


Figure 24. *Host: App-A | Recovery after restoring the rule | Result: ping to 10.20.1.10 returns 4 replies, 0% packet loss (earlier lines in the screenshot show the same after recovery).*

Repeat run

```
PS C:\Users\glenn>
PS C:\Users\glenn> ssh glens-app-a
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Thu Oct 29 04:05:41 2026 from 10.10.1.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3086ms
```

Figure 25. *Host: App-A | Repeat of the failure test: ping -c 4 10.20.1.10 | Result: 4 transmitted, 0 received, 100% packet loss.*

```
PS C:\Users\glenn> ssh glens-app-a
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Thu Oct 29 05:37:43 2026 from 10.10.1.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.212 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.218 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=4.58 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.239 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3095ms
rtt min/avg/max/mdev = 0.212/1.311/4.578/1.885 ms
[ec2-user@ip-10-10-2-10 ~]$
```

Figure 26. *Host: App-A | Repeat after recovery: ping -c 4 10.20.1.10 | Result: 4 transmitted, 4 received, 0% packet loss.*

Explanation: peering and both routes were unchanged during the test, so the network path was fine. Removing the Echo Request rule made the App-B security group drop the ICMP packets, so replies stopped. Restoring the rule brought replies back. This shows that routes decide where traffic can go, and security groups decide whether it is allowed.

10. Forward path and return path

Forward path (App-A to App-B)

- App-A (10.10.2.10) sends a packet to 10.20.1.10.
- The private route table of VPC A (glens-rt-a-private) matches 10.20.0.0/16 and sends it to the peering connection pcx-04a11b67292ed8e28.
- The packet arrives in VPC B. The App-B security group allows ICMP Echo Request from 10.10.2.10/32, so App-B receives it.

Return path (App-B to App-A)

- App-B replies to 10.10.2.10.
- The private route table of VPC B (glens-rt-b-private) matches 10.10.0.0/16 and sends the reply through the peering connection back to VPC A.
- Security groups are stateful, so the reply is allowed automatically. If the return route were missing, the reply would be dropped and the ping would fail even though peering is Active.

Administration path (SSH)

- Laptop to the bastion on its public IP (34.227.197.139), allowed by glens-sg-bastion and routed by the internet gateway.
- The bastion opens a second SSH connection to App-A (10.10.2.10) over the local VPC route, or to App-B (10.20.1.10) over the peering route.
- The destination sees the bastion's private IP 10.10.1.10 as the SSH source, so the app security groups only need to allow 10.10.1.10/32.

Issues I faced and how I fixed them

Problem	Cause	Fix
Subnet 10.10.1.0/24 was rejected: CIDR not within VPC CIDR	The first VPC had been created with 10.10.0.0/24 instead of /16, and the console had the default VPC selected	Deleted the empty VPC, recreated it with 10.10.0.0/16 and selected the correct VPC in the subnet form
Route to the internet gateway failed: route table and gateway belong to different networks	A route table was created in the default VPC instead of glens-vpc-a	Deleted it and recreated the route table in vpc-0847b1f0a36e6ac50, then added the route
Internet gateway showed no name	The tag key was not Name	Corrected the tag key to Name

11. Cleanup evidence

After saving all evidence, every resource created for this project was removed, in dependency order: terminate the instances; delete the peering routes and the peering connection; delete the security groups, subnets and custom route tables; detach and delete the internet gateway; delete both lab VPCs.

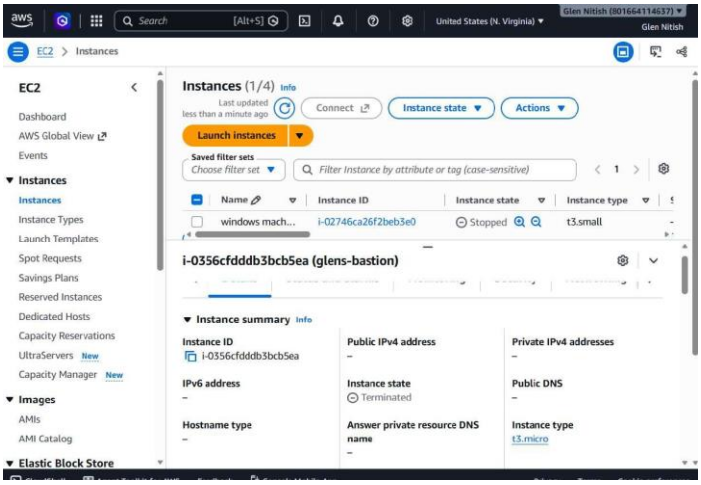


Figure 27. *glens-bastion (i-0356cfdddb3bcb5ea) - Instance state: Terminated.*

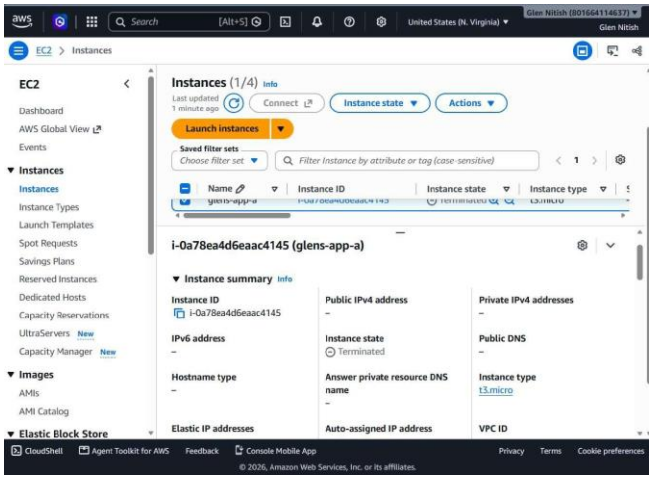


Figure 28. *glens-app-a (i-0a78ea4d6eaac4145) - Instance state: Terminated.*

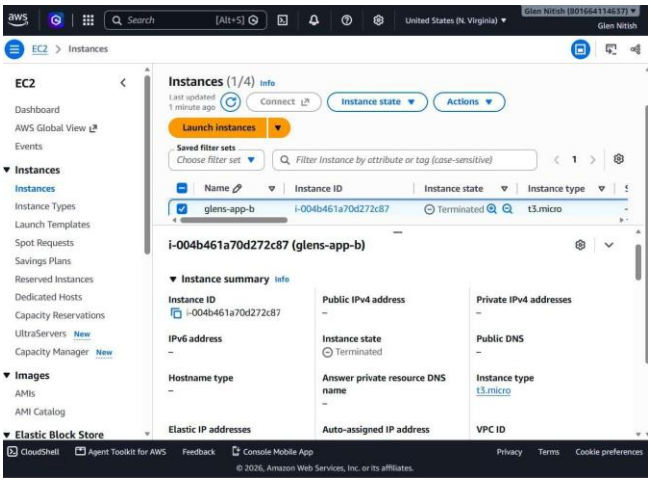


Figure 29. *glens-app-b (i-004b461a70d272c87) - Instance state: Terminated.*

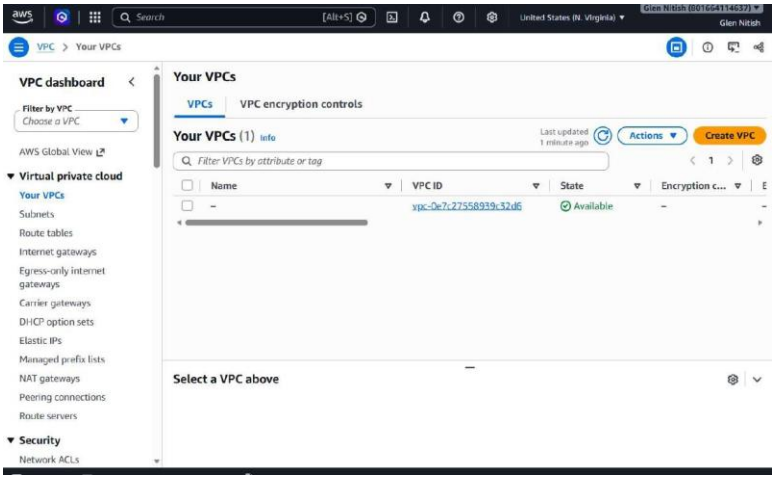


Figure 30. *VPC console after cleanup - only the default VPC remains (glens-vpc-a and glens-vpc-b are deleted).*

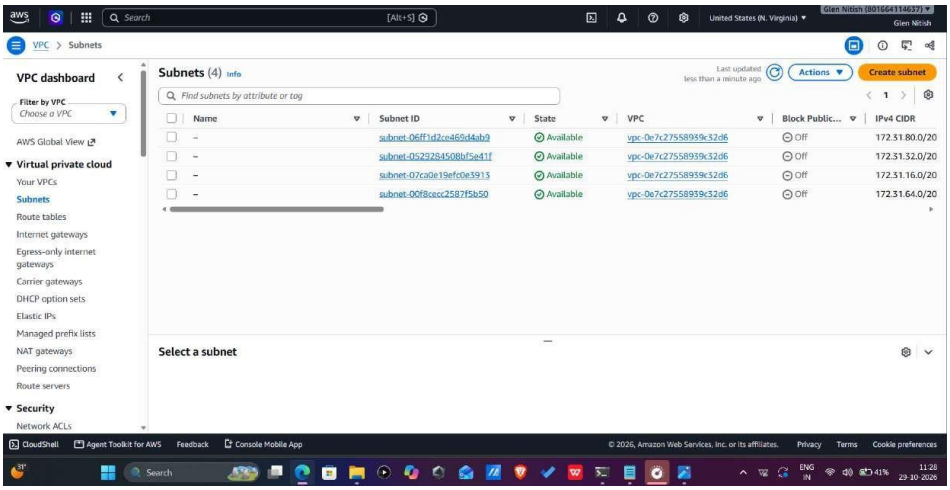


Figure 31. *Subnets after cleanup - only the four default-VPC subnets remain; all lab subnets are deleted.*

The default VPC and the separate Windows instance from earlier were not part of this project and were left untouched.

12. What I learned

I learned how to plan non-overlapping CIDRs and build public and private subnets with explicit route table associations. I saw that a subnet is public because its route table has a route to an internet gateway, not just because the gateway exists. I set up VPC peering and learned that Active status is not enough: both sides need routes, and the security groups must allow the traffic. I used a bastion host with SSH ProxyJump so the private servers stay private and the key never leaves my laptop. The controlled failure showed how to find which layer is blocking traffic, and the mistakes I made with CIDRs and route tables taught me to check the VPC ID on every resource I create.